

CMP722: Advanced Computer Vision – Project Proposal

Title: Comparative Analysis of Computer Vision Architectures for Reinforcement Learning Agents in ViZDoom

Group Members:

1. Mehmet Yusuf Bilge N25123057

1. Motivation & Problem Statement

The goal of this project is to explore the relationship between computer vision model architectures and the performance of reinforcement learning agents in Doom. Specifically, I aim to use ViZDoom, a first-person shooter simulation, to test how different deep learning models (CNNs, YOLO, etc.) perform when used as visual perception modules for an RL agent.

Traditional reinforcement learning agents in ViZDoom often rely on simple convolutional encoders to interpret pixel input. However, with the advancement of deep learning-based computer vision models (YOLO, etc.), there is potential to enhance an agent's understanding of the scene and improve learning efficiency.

This project will compare different deep learning architectures within the same reinforcement learning framework to measure their impact on performance.

2. Objectives

- Implement and compare multiple deep learning-based computer vision architectures (CNN, YOLO, etc.) for perception in ViZDoom.
- Integrate each model with the same reinforcement learning algorithm (e.g., PPO).
- Train agents to perform navigation or combat tasks in ViZDoom using visual input.
- Evaluate and compare models based on:
 - Detection accuracy / scene understanding (mAP, IoU, etc.)
 - RL performance metrics (reward per episode, learning speed)
- Analyze the trade-offs between model complexity, training time, perception accuracy, and gameplay performance.

3. Methodology

3.1 Problem Definition & Dataset

Task: Object detection and agent training in ViZDoom (e.g., detecting enemies, health packs, weapons).

Environment: ViZDoom API.

Dataset: Frames extracted from gameplay, optionally annotated for supervised pretraining.

Preprocessing: Image resizing, normalization, frame skipping, and data augmentation.

3.2 Model Development

- **Models:**
 - CNN-based encoder (e.g., 3-layer ConvNet).
 - YOLO for object detection.
 - As time permits, additional and more advanced models.
- **Frameworks:** PyTorch + ViZDoom + Stable-Baselines3 (for RL).
- **Training:**
 - Each vision model will process screen frames and output embeddings or detections.
 - The same RL algorithm (e.g., PPO) will use these outputs as input for policy learning.
 - Hyperparameters: learning rate, batch size, and exploration policy kept constant for fair comparison.

3.3 Evaluation & Analysis

- **Vision Metrics:** mAP, IoU, precision/recall on detection results.
- **Reinforcement Learning Metrics:** Average reward per episode, convergence rate, survival time.
- **Comparative Analysis:**
 - Evaluate how each visual model impacts agent performance.
 - Discuss efficiency, interpretability, and training stability.
 - Analyze potential reasons for performance gaps between architectures.

3.4 Demonstration

- Demo will showcase agents using different computer vision models playing ViZDoom.
- Visualization of:
 - Detected objects (bounding boxes).
 - Live reward graph.
 - Side-by-side gameplay of agents with CNN, YOLO, etc. models.

4. Expected Outcomes

- Trained RL agents integrated with multiple computer vision model architectures.
- A demo video showing agents' gameplay and detection visualizations.
- Insights into how perception quality influences reinforcement learning in visual environments.